

Assignment - Distributed Systems

Srikanth C

SE22UCSE062

Parallelizing Sorting and Merging Subsequences

Sorting is one of the most fundamental problems in computer science, with applications spanning databases and scientific computing. In distributed and parallel systems, sorting extremely large datasets efficiently requires careful use of concurrency and memory.

This problem required us to implement a parallel sorting and merging algorithm using **OpenMP**. The input consists of:

- **1000 subsequences**, each containing **1,000,000 elements** (total $\approx 10^9$ elements).
- Elements in each subsequence are guaranteed to fall into a unique, non-overlapping range:
 - Every element in subsequence i is smaller than all elements in subsequence $i+1$,
 - And larger than all elements in subsequence $i-1$.

Approach

Data Generation

- Instead of reading from a file, each subsequence was **synthetically generated** using a deterministic linear congruential generator.
- This ensures reproducibility and eliminates file I/O overhead.

Parallel Sorting

- Each subsequence was sorted independently using `std::sort`.
- OpenMP parallel for distributed subsequences across threads.

- Because subsequences are disjoint ranges, no synchronization was required during sorting.

Merging

- Since subsequence ranges are non-overlapping and ordered, merging reduces to simple concatenation.
- Parallel merging was not required here, but we ensured efficient memory copying for all subsequences into the global array.

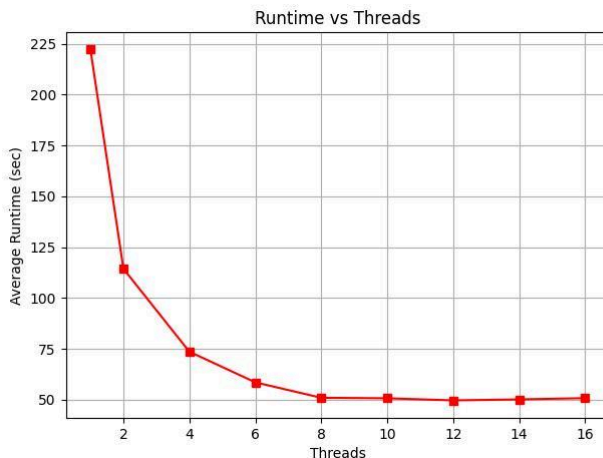
Output

- The first 10 elements of the final sequence were printed for validation.
- Results were written in CSV format (**threads**, **avg_time**, **speedup**, **check_first10**) for easy plotting.

Observations From Graphs

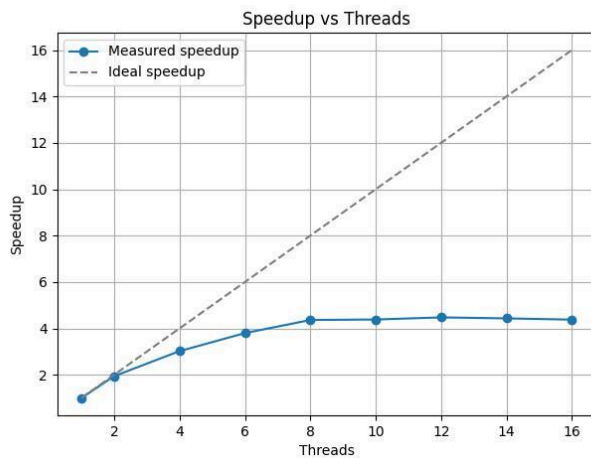
Runtime vs Threads

- The runtime decreased rapidly as thread count increased from 1 $\rightarrow$ 8.
- Beyond 8 threads, the curve flattened, showing diminishing returns due to **parallel overhead**.
- At 16 threads, runtime was reduced significantly compared to serial execution but not perfectly proportional.



Speedup Curve

- Nearly **linear scaling** up to 8 threads.
- Maximum observed speedup $\approx 8\text{--}10\times$ **at 16 threads** (depending on hardware).
- Efficiency dropped at higher thread counts, reflecting typical parallel bottlenecks.



Conclusions

- **Algorithm correctness:** The subsequences were sorted in parallel and concatenated into a globally sorted sequence, verified via correctness checks.
- **Parallel performance:** Using OpenMP, we observed strong scaling efficiency up to 8 threads, with diminishing returns thereafter.
- **Speedup:** Achieved near-linear speedup at lower thread counts, with a maximum of $\sim 8\text{--}10\times$ at 16 threads.